

Claude Code như "Super Orchestrator" — Guide sơ bộ cho người học

Dành cho: học viên 4 track (PM / BA / SA / Dev) của AI Bootcamp. **Bổ trợ cho:** Program + 4 workbook nghề + **Operating Model**. **Ý chính:** đừng coi AI coding agent là "chatbot trả lời câu hỏi" — hãy dùng nó như **một nhạc trưởng (orchestrator)** đứng ra **làm trọn từng bước công việc**: đọc bối cảnh → lập kế hoạch → tự gọi công cụ/chạy lệnh → tự kiểm → ghi log — còn **bạn là người chỉ huy + phán xử + chịu trách nhiệm**. **Áp dụng cho:** Claude Code, Codex, Cursor, hay bất kỳ AI coding agent nào — nguyên tắc điều phối là **tool-agnostic**, chỉ cách bật tính năng cụ thể khác nhau giữa các tool. **Thuật ngữ:** tra cứu đầy đủ + mapping ngành tại [docs/GLOSSARY.md](#). **Ngày:** 2026-08-24 · **Phiên bản:** v2.0

o. Tóm tắt 1 phút — "Orchestrator" nghĩa là gì

Một chatbot thường: bạn hỏi 1 câu → nó trả 1 đoạn văn → bạn tự đi làm phần còn lại.

AI coding agent như orchestrator: bạn giao cả một mục tiêu nhiều bước → nó tự phân rã, tự đọc file, tự viết file, tự chạy lệnh, tự gọi công cụ ngoài, tự sinh "trợ lý con" (subagent) chạy song song, tự kiểm tra kết quả, tự sửa — rồi báo lại. Bạn không gõ từng lệnh nhỏ; bạn đặt mục tiêu, đặt lan can (ranh giới), duyệt ở cổng, và bắt lỗi.

 **Tool-agnostic:** guide này lấy Claude Code làm ví dụ, nhưng nguyên tắc (vòng 7 nhíp, Leash A/A+, fail-closed gate) áp dụng cho **mọi AI coding agent** — Codex, Cursor, Windsurf... Cái thay đổi giữa các tool là cách bật tính năng (plan mode, subagent, hook); cái không đổi là **tư duy điều phối + governance** của bạn.

 **Ẩn dụ:** bạn là **nhạc trưởng**, Claude Code là **cả dàn nhạc + người điều phối bè**. Bạn không tự chơi từng nhạc cụ — bạn ra tổng phổ, chỉ nhịp, và nghe ra chỗ sai để chỉnh. Đây đúng là tinh thần **cặp (người + AI)** và **CASAN Cấp 2 → 4**.

Vì sao học điều này: cả khoá xoay quanh việc **tự tay lái AI build một thứ chạy được** (Customer Zero). Claude Code là "cỗ máy" để làm điều đó cho **mọi bước** — từ viết requirement, dựng kiến trúc, sinh code, tới chạy test và log telemetry.

1. Đổi tư duy: từ "hỏi–đáp" sang "giao–điều phối"

Kiểu cũ (chatbot)	Kiểu orchestrator (Claude Code)
Hỏi 1 câu, copy câu trả lời ra ngoài	Giao mục tiêu, để nó làm thẳng trên file/dự án
Bạn tự ghép các bước	Nó tự lập kế hoạch nhiều bước rồi thực thi
Kết quả nằm trong khung chat	Kết quả là artefact thật (file, code chạy, test xanh)
Bạn tin hoặc không tin	Bạn đặt cổng kiểm + tự verify trước khi dùng
1 việc 1 lúc	Nhiều subagent chạy song song cho việc lớn

Câu thần chú: "Đừng bảo nó viết cho tôi một đoạn — hãy bảo nó hoàn thành cả bước, rồi tôi kiểm."

2. Vòng lặp Orchestrator chuẩn (7 bước) — dùng cho MỌI bước công việc

Áp cho bất kỳ task nào (viết requirement, dựng kiến trúc, code, test...):

- Context** — **nạp bối cảnh**. Cho nó biết "sân chơi": file `CLAUDE.md` mô tả dự án, tài liệu liên quan, ràng buộc. *Bối cảnh tốt = kết quả tốt.*
- Plan** — **bắt nó lập kế hoạch trước**. Yêu cầu nó **trình kế hoạch từng bước** và **chờ bạn duyệt** rồi mới làm (dùng **Plan mode**). Bạn sửa kế hoạch ở đây rõ hơn sửa kết quả sau.
- Delegate** — **giao có ranh giới**. Nói rõ **được làm gì / không được làm gì** (Leash **A** = tới bản nháp, không tự đẩy/merge; **A+** = việc rủi ro, phải chờ bạn duyệt).
- Execute** — **để nó chạy trọn bước**. Nó tự đọc/ghi file, chạy lệnh, gọi công cụ, sinh subagent. Bạn quan sát, không micromanage.
- Gate/Verify** — **cổng kiểm mặc-định-đóng**. Trước khi dùng kết quả: chạy test/lint/kiểm chứng. **Không xác minh được = chặn**, không "tạm cho qua". Bạn tự đọc-hiểu, không ký bừa.
- Log** — **ghi ngay**. Ghi **Dev Book** (chỗ AI sai → bạn sửa) + telemetry (giờ thật, số lần bạn sửa AI). *Ghi cùng nhịp, không để sau.*
- Iterate** — **lặp lại** cho bước tiếp theo.

Đây chính là vòng **Context → Plan → Delegate → Execute → Gate → Log → lặp**. Thuộc nó là thuộc nghề.

3. "Cần điều khiển" của Claude Code — 8 thứ nên biết sớm

Không cần giỏi kỹ thuật; biết **khi nào bật cái nào**:

Cần điều khiển	Là gì	Khi nào dùng
Context file (CLAUDE.md / .codex / tương đương)	File "trí nhớ dự án" — mô tả bối cảnh, quy ước, việc cần tuân	Đặt ở gốc dự án; nạp bối cảnh 1 lần cho mọi phiên
Plan mode	Chế độ "lập kế hoạch trước, chờ duyệt rồi mới làm"	Việc nhiều bước / rủi ro → luôn plan trước
Subagent / chạy song song	Sinh nhiều "trợ lý con" làm nhiều việc cùng lúc	Việc lớn: rà nhiều file, làm nhiều nhánh song song
MCP tools	Cắm công cụ ngoài (trình duyệt, DB, API, tài liệu)	Khi cần đụng hệ thống thật, không chỉ văn bản
Skills / lệnh / ...	Kỹ năng đóng gói sẵn (tạo tài liệu, review code, v.v.)	Gõ / để xem; dùng cho tác vụ chuẩn hoá
Hooks	Tự chạy 1 lệnh sau mỗi hành động (vd tự kiểm)	Dựng "lan can tự động" — vd tự chạy test sau khi sửa
Memory	Ghi nhớ sở thích/quyết định qua nhiều phiên	Để nó nhớ cách bạn muốn làm việc
Permission mode	Nấc quyền: hỏi từng bước ↔ tự làm trong ranh giới	Đặt đúng "dây cương" A/A+ cho từng loại việc

💡 **Non-coder yên tâm:** bạn **không cần nhớ cú pháp**. Kẹt chỗ nào → **hỏi thẳng Claude Code** ("giải thích lỗi này", "làm sao bật plan mode") — nó là **mentor trực mặc định** của bạn.

4. Áp vào từng bước — Claude Code làm gì trong mỗi workbook nghề

Cùng một orchestrator, mỗi nghề giao một loại "vật phẩm":

Bước / Bài	Giao cho Claude Code làm trọn bước	Bạn (người) phán xử
BA — Requirement/Story	Đọc brief → sinh requirement có cấu trúc → tự review tìm mâu thuẫn → sinh User Story + AC (Gherkin)	Lọc câu hỏi thật vs AI-tự-sai; bổ case biên; hard-stop chỗ mơ hồ
BA — Prototype	Dựng prototype HTML/clickable chạy được cho 1 luồng	Tự bấm thử, đối chiếu AC, ghi chỗ lệch

Bước / Bài	Giao cho Claude Code làm trọn bước	Bạn (người) phản xử
SA — Kiến trúc/ADR	Sinh 2–3 phương án + sơ đồ + bảng trade-off	Chọn phương án, viết ADR, bắt chỗ over/under-engineer
SA — Slice kiến trúc	Dựng skeleton chạy được đúng ranh giới thành phần	Kiểm ranh giới đúng ADR; nấn chỗ AI trộn tăng
Dev — Code + Test	Sinh code từ spec → sinh unit/integration test	Review code như review PR ; loại "test giả"; gài bug thử
Dev — Harness/agent	Dựng 1 agent nhỏ (context·tool·gate) tự làm tác vụ lặp	Đặt cổng fail-closed; thử cho nó sai để xác nhận bị chặn
PM — WBS/Estimation/Risk	Breakdown scope → WBS → ước lượng → risk register	Bắt task bỏ sót; chỉnh ước lượng theo team thật
Mọi nghề — Weekly/Dashboard	Sinh báo cáo + dashboard từ dữ liệu thật	Kiểm số không bịa; rút 1 quyết định
Mọi nghề — Delegation Map	Đề xuất mức L0–L5 + A/A+ cho từng task	Không để nó tự nâng mức việc rủi ro cao

Điểm chung: Claude Code **làm phần đóng-hộp-được**; bạn giữ **phần phán đoán** (chọn, sửa, dừng, chịu trách nhiệm).

5. Governance overlay — lái orchestrator mà vẫn an toàn

Đây là chỗ **nhấn mạnh nhất** — orchestrator mạnh thì **lan can phải chắc**:

- **Leash A (mặc định)**: Claude Code tới **bản nháp / bản đề xuất**, **KHÔNG** tự **đẩy/merge/phát hành**. Đây là chế độ chạy hằng ngày.
- **Leash A+ (việc rủi ro)**: đụng dữ liệu nhạy cảm, đổi schema DB, phân quyền, tích hợp hệ thật → **bắt buộc bạn đích thân duyệt** (BUILT-flagged: nó làm xong nhưng **chờ duyệt** mới dùng).
- **Fail-closed gate**: đặt cổng kiểm (test/lint/security). **Không xanh = không đi tiếp**. Có thể tự động bằng **hooks**.
- **Hard-stop**: khi spec **mâu thuẫn / thiếu để quyết**, yêu cầu nó **DỪNG và hỏi**, tuyệt đối không tự đoán. *AI dừng lại hỏi là TỐT, không phải lỗi*.
- **Human-in-the-loop**: artefact quan trọng → **người đọc-hiểu thật** trước khi ký. Đây là ranh giới chống **rubber-stamping**.

 **Quy tắc vàng:** "Càng để AI tự chạy nhiều bước, cổng kiểm càng phải chặt."
Orchestrator không phải để bạn buông tay — mà để bạn **rảnh tay lo phần phán đoán**.

6. Telemetry + Dev Book — ghi NGAY trong lúc lái

Vừa lái vừa ghi (đây là bằng chứng bạn **hiểu**, không rubber-stamp):

- **Dev Book (nhật ký quyết định):** mỗi bước ghi — *chỗ AI làm sai* → *bạn sửa gì* · *vì sao chọn mức L này* · *cổng nào chặn* · *hard-stop nào gặp*. Chấm **nặng hơn cả bản kế hoạch**.
- **Telemetry tay-làm:** giờ ngồi máy **thật** (không suy từ token) · **số lần bạn sửa/override AI** · token ghi ước tính rồi đối soát sau.

Một người có 30 lần sửa AI ≠ một người có 0 lần sửa — kẻ thứ hai đang rubber-stamp.
Con số "số lần sửa" là cột **đắt nhất**.

7. Quickstart "ngày đầu tiên" — 10 bước làm được ngay

1. Mở dự án (hoặc chọn 1 đề PB-01→PB-06) trong Claude Code.
2. Tạo/đọc CLAUDE.md — cho nó biết dự án là gì, quy ước, ràng buộc.
3. Giao mục tiêu bước đầu bằng **1 câu mục tiêu** (không phải 10 lệnh nhỏ).
4. Bật **Plan mode** → đọc kế hoạch nó trình → **sửa kế hoạch** cho đúng ý.
5. Duyệt kế hoạch → để nó **thực thi trọn bước**.
6. Khi nó chạm việc rủi ro (đổi DB, xóa file, gọi hệ thật) → **dừng lại duyệt** (A+).
7. Kết quả xong → **tự verify**: chạy test / bấm thử / đọc-hiểu. Không xanh → bảo nó sửa.
8. Bắt được ≥1 chỗ AI sai → ghi vào **Dev Book** + sửa.
9. Ghi **telemetry** (giờ thật + số lần sửa).
10. Sang bước tiếp → lặp lại vòng 7 bước (§2).

Làm hết 10 bước này 1 lần = bạn đã chạy trọn vòng orchestrator cho 1 task thật.

8. Prompt patterns mẫu (để lái, không phải để hỏi)

- **Giao trọn bước:** "Mục tiêu: [X]. Hãy tự lập kế hoạch các bước, chờ tôi duyệt rồi thực thi trên file dự án. Nếu giả định thay vì bịa."

- **Bắt lập kế hoạch trước:** "Chưa làm vội. Trình cho tôi kế hoạch từng bước + rủi ro + chỗ cần tôi quyết. Tôi duyệt rồi hãy chạy."
- **Cắm ranh giới (leash):** "Được sửa code + chạy test. KHÔNG được commit/đẩy/đổi schema DB khi chưa hỏi tôi."
- **Ép hard-stop:** "Nếu có chỗ nào mâu thuẫn hoặc thiếu thông tin để quyết, DỪNG và liệt kê câu hỏi — đừng tự đoán."
- **Cổng kiểm:** "Sau khi sửa xong, tự chạy test + lint và báo kết quả. Nếu đỏ, sửa tới khi xanh rồi mới coi là xong."
- **Việc lớn → song song:** "Việc này lớn, chia thành các phần độc lập và chạy song song bằng nhiều subagent, rồi tổng hợp lại cho tôi."
- **Tự phản biện:** "Đóng vai reviewer khó tính, tìm lỗi/thiếu sót trong chính kết quả bạn vừa tạo."

9. Bẫy thường gặp + chống rubber-stamping

Bẫy	Hậu quả	Cách tránh
Copy output AI ra mà không đọc	Lỗi lặt, "tốt nghiệp" mà không hiểu	Luôn tự verify + ghi ≥1 chỗ đã sửa
Cho tự chạy việc rủi ro không cổng	Mất dữ liệu, lộ thông tin	Việc rủi ro → A+ chờ duyệt + gate
Prompt mơ hồ, thiếu bối cảnh	Nó đoán → sai hướng	Nạp <code>CLAUDE.md</code> + mục tiêu rõ + Plan mode
Tin số AI tự bịa	Báo cáo/estimate sai	Bắt nêu nguồn; không bịa ngoài dữ liệu cho
Micromanage từng lệnh	Mất hết lợi thế orchestrator	Giao mục tiêu , không giao thao tác
Không ghi Dev Book/telemetry	Không có bằng chứng hiểu	Ghi ngay trong lượt , không để sau

⚠ **Lằn ranh đỏ của cả khoá:** nộp thứ AI làm mà không có dấu vết bạn đã review/sửa = **rubber-stamping** = **trượt**, dù kết quả "đẹp" hay "chạy được".

10. Checklist bỏ túi (in ra dán cạnh màn hình)

Trước khi giao việc: - [] Đã nạp bối cảnh (`CLAUDE.md` + tài liệu)? - [] Đã nêu **mục tiêu** (không phải thao tác)? - [] Đã bật **Plan mode** cho việc nhiều bước? - [] Đã **cắm ranh giới A/A+** (được gì / không được gì)?

Trong khi chạy: - [] Việc rủi ro có **dừng để mình duyệt** không? - [] Có chỗ spec mơ hồ → đã **hard-stop** chưa?

Trước khi coi là xong: - ☐ Đã **tự verify** (test/bấm thử/đọc-hiểu)? - ☐ Đã bắt & sửa **≥1 chỗ AI sai**? - ☐ Đã ghi **Dev Book + telemetry** ngay?

Thuộc 3 khối checklist này = bạn đang dùng Claude Code đúng như một **super orchestrator có kiểm soát** — không phải một cái máy đoán chữ.

Guide này là tài liệu nhập môn. Đi kèm: Program (§5 Mindset, §8 Capstone), 4 workbook nghề (bài tập cụ thể), và Operating Model (L0–L5 · Leash A/A+ · 3 cơ chế governance). Tra thuật ngữ: `docs/GLOSSARY.md`.